

Problem A. 2090 Virus

Input file: **standard input**
Output file: **standard output**
Time limit: 1 second
Memory limit: 1024 megabytes

In the year 2090 on Planet Blue, a meme virus has swept across the world. Every person infected with this virus can only speak in “meme language”. A sentence is represented as a string s consisting only of lowercase English letters. It is classified as meme language if and only if all the following conditions are met:

- The length of s is exactly 8.
- The characters at positions 1, 3, 5, and 7 are consonants (i.e., they are not ‘a’, ‘e’, ‘i’, ‘o’, or ‘u’).
- The characters at positions 2, 4, 6, and 8 are vowels (‘a’, ‘e’, ‘i’, ‘o’, or ‘u’).

For example, “bababoyi” and “bibilabu” are valid meme language strings. However, “wodedaodun” is not (the length is not 8), “aiyogama” is not (the 1-st character ‘a’ is not a consonant), and “ruheneng” is not (the 8-th character ‘g’ is not a vowel).

You are the person in charge of a shelter on Planet Blue, tasked with identifying survivors who have not been infected. To determine infection status, you ask each person to say a sentence. If the sentence is not in “meme language”, the person is confirmed to be uninfected (a survivor). Otherwise, the person is considered to be infected.

Given the sentences spoken by n people, determine whether each person is a survivor.

Input

The first line contains one integer n ($1 \leq n \leq 10^5$), representing the number of people to be checked.

Each of the next n lines contains a string s consisting only of lowercase English letters, representing the sentence spoken by a person.

It is guaranteed that the total length of all strings s does not exceed 10^6 .

Output

Output n lines. For the i -th line, if the i -th person’s sentence is meme language, output “Suspected Virus” (without quotes); otherwise, output “Well-Being” (without quotes).

Example

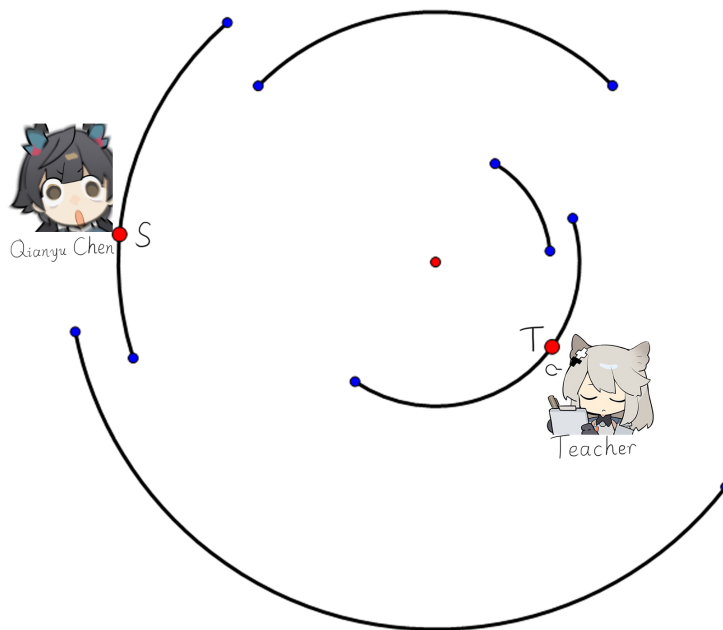
standard input	standard output
6	Suspected Virus
bababoyi	Suspected Virus
bibilabu	Well-Being
wodedaodun	Well-Being
aiyogama	Well-Being
ruheneng	Suspected Virus
hehehahi	

Problem B. Cast off as Cast

Input file: standard input
Output file: standard output
Time limit: 3 seconds
Memory limit: 1024 megabytes

Class is starting, but Qianyu Chen is still in bed!

In the top-down view of the school, the locations of the dormitory and the classroom are situated among n concentric circular arcs with distinct radii. Each arc represents a building, defined by its radius r_i and its angular range $\langle \theta_{i,1}, \theta_{i,2} \rangle$.



The movement rules are as follows: A person can move along any circular arc (circumferential direction) or along any radial line (straight line toward or away from the center). However, moving through or stopping at the exact center ($r = 0$) is strictly prohibited. If a position is denoted by (r, θ) , where r is the radial distance to the center, and θ is an angle in radians measured counterclockwise from the positive x -axis, the movement rules are formalized as follows:

- A person can move from (r, θ_1) to (r, θ_2) if and only if there exists a building arc with radius r such that the entire angular path from θ_1 to θ_2 (or vice versa) lies within the angular range of that arc;
- A person can move from (r_1, θ) to (r_2, θ) along a straight radial line if and only if $r_1 > 0$ and $r_2 > 0$ (i.e., continuous radial movement is allowed between any two non-zero radii at the same angle θ).

Qianyu Chen starts from the dormitory at (r_s, θ_s) and needs to reach the classroom at (r_t, θ_t) . Calculate the shortest distance required. If the classroom is unreachable, output **“Cast off as cast!”**.

Input

The first line contains five integers n ($1 \leq n \leq 10^5$), r_s , d_s , r_t , and d_t ($1 \leq r_s, r_t \leq 10^5$, $0 \leq d_s, d_t < 10^5$), representing the number of buildings, the dormitory position, and the classroom position. The angle θ for any position given by an integer d is computed via $\theta = \frac{2\pi d}{10^5}$ radians.

The next n lines each contain three integers r_i , $d_{i,1}$, and $d_{i,2}$ ($1 \leq r_i \leq 10^5$, $0 \leq d_{i,1}, d_{i,2} < 10^5$), describing a building arc. The angular range $\langle \theta_{i,1}, \theta_{i,2} \rangle$ is interpreted counterclockwise as follows:

- If $d_{i,1} \leq d_{i,2}$, the arc extends counterclockwise continuously from $\theta_{i,1} = \frac{2\pi d_{i,1}}{10^5}$ to $\theta_{i,2} = \frac{2\pi d_{i,2}}{10^5}$ radians. When equal, it represents a single point;
- If $d_{i,1} > d_{i,2}$, the arc extends counterclockwise from $\theta_{i,1} = \frac{2\pi d_{i,1}}{10^5}$ to 2π (the positive x -axis), and then immediately continues from 0 to $\theta_{i,2} = \frac{2\pi d_{i,2}}{10^5}$ radians.

It is guaranteed that all r_i are distinct, and both the dormitory and the classroom lie on some building.

Output

If the classroom is unreachable, output “**Cast off as cast!**” (without quotes) in a single line. Otherwise, output a single real number representing the shortest distance in a single line.

Your answer is acceptable if its absolute or relative error does not exceed 10^{-6} . Formally, let your output be a and the jury's answer be b . Your output is accepted if and only if $\frac{|a-b|}{\max(1,|b|)} \leq 10^{-6}$.

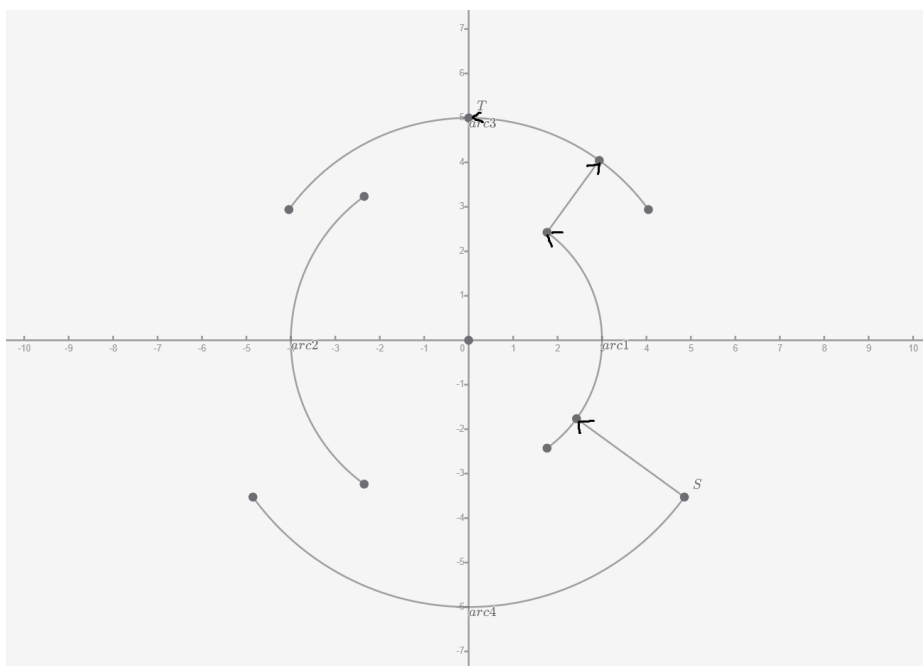
Examples

standard input	standard output
4 6 90000 5 25000 3 85000 15000 4 35000 65000 5 10000 40000 6 60000 90000	12.853981633974
2 1 0 2 1 1 0 0 2 1 1	Cast off as cast!

Note

For the first sample case, as shown in the figure, the shortest path consists of two radial segments and two arc segments. The total distance is:

$$\underbrace{3}_{\text{radial}} + \underbrace{\left(\frac{25000 \cdot 2\pi}{10^5}\right) \cdot 3}_{\substack{\text{arc at } r=3 \\ \Delta d=25000}} + \underbrace{2}_{\text{radial}} + \underbrace{\left(\frac{10000 \cdot 2\pi}{10^5}\right) \cdot 5}_{\substack{\text{arc at } r=5 \\ \Delta d=10000}} = 5 + \frac{5\pi}{2} \approx 12.853981633974$$



Problem C. Fish Eating

Input file: **standard input**
Output file: **standard output**
Time limit: 4 seconds
Memory limit: 1024 megabytes

Given an $n \times m$ grid where some cells contain fish and the remaining cells are obstacles. One of the fish on the grid is under your control, and you may repeatedly move it to any adjacent cell (up, down, left, or right), but cannot move outside the grid: if the target cell contains a fish of size no greater than its own, it may eat that fish and its size increases by 1; if the target cell is an obstacle or contains a fish strictly larger, it cannot move there.

Initially, every cell is an obstacle. There are q operations in order, of the following two types:

1. Remove the obstacle at (x, y) , and place a fish of size v there. Find the maximum number of fish this fish can eat under your control. It is guaranteed that (x, y) is currently an obstacle, and that v is not smaller than the sizes of all previously placed fish.
2. Assume that before the hypothetical process, you may increase the initial size of the fish at (x, y) by a non-negative integer. Let k be the maximum number of fish it can possibly eat among all choices of enlargement. Find the minimum enlargement needed to eat k fish. It is guaranteed that a fish has been placed at (x, y) .

For both types of operations, the eating process is hypothetical and does not change the actual state of any placed fish.

Input

The first line of the input contains three integers n, m ($1 \leq n, m, n \times m \leq 2.5 \times 10^5$), and q ($1 \leq q \leq 5 \times 10^5$), representing the dimensions of the grid and the number of operations.

You need to process q operations in order. For each operation, you are given encrypted coordinates x' and y' . Let l be the output of the previous operation (for the first operation, $l = 0$). Calculate the decrypted coordinates $x = x' \oplus l$, $y = y' \oplus l$, where $\oplus$ denotes bitwise XOR. It is guaranteed that $1 \leq x \leq n$ and $1 \leq y \leq m$. The value v in type 1 operations is not encrypted. The operations are:

1. 1 x' y' v : Calculate the decrypted coordinates x and y , remove the obstacle at (x, y) , and place a fish of size v ($1 \leq v \leq 10^9$) there. Find the maximum number of fish this fish can eat under your control. It is guaranteed that (x, y) is currently an obstacle, and that v is not smaller than the sizes of all previously placed fish.
2. 2 x' y' : Calculate the decrypted coordinates x and y . Assume that before the hypothetical process, you may increase the initial size of the fish at (x, y) by a non-negative integer. Let k be the maximum number of fish it can possibly eat among all choices of enlargement. Find the minimum enlargement needed to eat k fish. It is guaranteed that a fish has been placed at (x, y) .

Output

Output q lines, each containing a single integer, representing the answer to each operation.

Example

standard input	standard output
2 3 9	0
1 1 2 1	0
1 2 1 1	2
1 2 2 2	1
2 3 0	3
1 0 2 8	5
2 1 2	4
1 4 4 9	4
2 6 6	0
2 5 7	

Note

We explain the sample test case below. After decrypting the coordinates:

1. 1 1 2 1: place a fish of size 1 at (1,2); it can eat at most 0 fish.
2. 1 2 1 1: place a fish of size 1 at (2,1); it can eat at most 0 fish.
3. 1 2 2 2: place a fish of size 2 at (2,2); it can eat at most 2 fish.
4. 2 1 2: the fish at (1,2) has size 1; it can eat at most 2 fish if allowed arbitrary enlargement, and must be enlarged by at least 1 to achieve this maximum.
5. 1 1 3 8: place a fish of size 8 at (1,3); it can eat at most 3 fish.
6. 2 2 1: the fish at (2,1) has size 1; it can eat at most 3 fish if allowed arbitrary enlargement, and must be enlarged by at least 5 to achieve this maximum.
7. 1 1 1 9: place a fish of size 9 at (1,1); it can eat at most 4 fish.
8. 2 2 2: the fish at (2,2) has size 2; it can eat at most 4 fish if allowed arbitrary enlargement, and must be enlarged by at least 4 to achieve this maximum.
9. 2 1 3: the fish at (1,3) has size 8; it can eat at most 4 fish if allowed arbitrary enlargement, and no enlargement is needed to reach this maximum.

Problem D. Something Different

Input file: **standard input**
Output file: **standard output**
Time limit: 2 seconds
Memory limit: 1024 megabytes

There is a special type of chocolate bar that can be regarded as an m -dimensional hyperrectangle, with side lengths in the ratio $s_1 : s_2 : \dots : s_m$, where $s_1, s_2, \dots, s_m$ are given positive even integers.

Alice and Bob plan to share this type of chocolate bar. First, they choose a positive integer n and then order such a chocolate bar with side lengths $s_1n, s_2n, \dots, s_mn$, which is placed on the table. They then take turns, with Alice going first, making one cut through the chocolate bar on the table to divide it into two hyperrectangular pieces, each with positive integer side lengths. After each cut, they take away one of the pieces and eat it, leaving the other piece on the table. They stop until neither can make any valid cut.

Formally, on a certain turn, suppose the remaining chocolate bar on the table has side lengths $l_1, l_2, \dots, l_m$. If $l_1 = l_2 = \dots = l_m = 1$, this final chocolate bar is discarded and not eaten by anyone. Otherwise, the current person chooses one dimension i ($1 \leq i \leq m$) with $l_i > 1$, and chooses an integer cut position k ($1 \leq k < l_i$). The chocolate bar is cut into two m -dimensional chocolate bars with side lengths $l_1, \dots, l_{i-1}, k, l_{i+1}, \dots, l_m$ and $l_1, \dots, l_{i-1}, (l_i - k), l_{i+1}, \dots, l_m$.

However, both Alice and Bob have cavities. Unable to resist sweets yet afraid that their teeth might decay further, they will each adopt the optimal strategy to make the total hypervolume (e.g., area for a two-dimensional chocolate bar, volume for a three-dimensional chocolate bar) of the chocolate bars they ultimately eat as small as possible. Each time a piece with side lengths $l_1, l_2, \dots, l_m$ is eaten, it contributes a value of $l_1 \times l_2 \times \dots \times l_m$ to this total.

For fixed n , let $A(n)$ be Alice's final consumed total hypervolume and $B(n)$ be Bob's final consumed total hypervolume. Your task is to compute the asymptotic leading coefficient of the difference $A(n) - B(n)$ as $n \rightarrow \infty$, modulo 998 244 353. More specifically, it can be proved that there exists a degree d and a non-zero rational number C such that for every real $\varepsilon > 0$, there exists an integer N such that for all integers $n \geq N$, $\left| \frac{A(n) - B(n)}{n^d} - C \right| < \varepsilon$. Namely, $C = \lim_{n \rightarrow \infty} \frac{A(n) - B(n)}{n^d}$. Since C can be denoted by an irreducible fraction $\frac{P}{Q}$, please output the integer R satisfying $0 \leq R < 998\,244\,353$ and $R \cdot Q \equiv P \pmod{998\,244\,353}$. It can be proved that such R exists and is unique.

Input

The first line contains one integer m ($2 \leq m \leq 5 \times 10^5$).

The second line contains m integers $s_1, s_2, \dots, s_m$ ($2 \leq s_i \leq 10^9$, s_i is even).

Output

Output the value of C modulo 998 244 353.

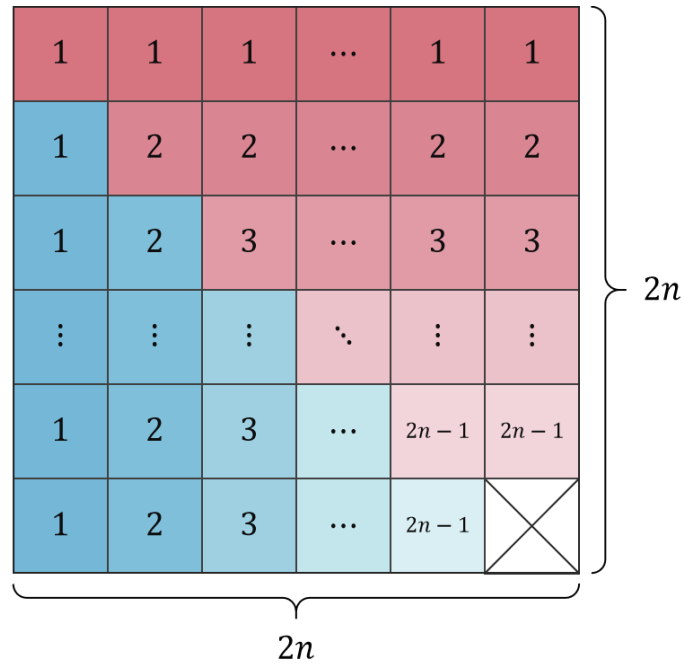
Examples

standard input	standard output
2 2 2	2
3 6 8 2	10

Note

For the first sample case, the chocolate bar is a $2n \times 2n$ square, so the hypervolume of each eaten piece is its area. The figure below shows one possible outcome under optimal strategies: Alice repeatedly cuts

off the topmost row, Bob repeatedly cuts off the leftmost column, and the final 1×1 piece is discarded. The colors indicate which unit cells are ultimately eaten by Alice and Bob, while the number in each cell indicates the round in which that cell is cut off.



Every non-diagonal cell eaten by Alice has a mirror cell across the main diagonal that is eaten by Bob, and the only extra cells are the $2n - 1$ diagonal cells eaten by Alice; hence $A(n) - B(n) = 2n - 1$, and its asymptotic leading coefficient is 2 since $C = \lim_{n \rightarrow \infty} \frac{2n-1}{n} = 2$.

Problem E. Permutation Evaluation

Input file: **standard input**
Output file: **standard output**
Time limit: 1 second
Memory limit: 1024 megabytes

For a permutation P of numbers from 0 to $n - 1$, we define its value as:

$$f(P) = \sum_{0 \leq i < j < n} (P_j - P_i)$$

Given a permutation P of numbers from 0 to $n - 1$, output its value.

Input

The first line contains an integer n ($1 \leq n \leq 2 \times 10^5$).

The second line contains n integers $P_0, P_1, \dots, P_{n-1}$, representing the given permutation of numbers from 0 to $n - 1$.

Output

Output a line containing an integer, the value of the permutation.

Example

standard input	standard output
4 2 0 1 3	4

Note

For the sample case:

$$\begin{aligned} f(P) &= (P_1 - P_0) + (P_2 - P_0) + (P_3 - P_0) + (P_2 - P_1) + (P_3 - P_1) + (P_3 - P_2) \\ &= (0 - 2) + (1 - 2) + (3 - 2) + (1 - 0) + (3 - 0) + (3 - 1) \\ &= -2 + (-1) + 1 + 1 + 3 + 2 \\ &= 4 \end{aligned}$$

Problem F. Permutation Generation

Input file: **standard input**
Output file: **standard output**
Time limit: 1 second
Memory limit: 1024 megabytes

For a permutation P of numbers from 0 to $n - 1$, we define its value as:

$$f(P) = \sum_{0 \leq i < j < n} (P_j - P_i)$$

Given a permutation P of numbers from 0 to $n - 1$, and two integers k and x ($0 \leq k, x < n$), construct a permutation P' of numbers from 0 to $n - 1$ such that the following two conditions are met:

- $P'_k = x$ (the element at index k in P' is x , using 0-based indexing);
- $f(P') \equiv f(P) \pmod{n}$.

If there is a solution, output any valid permutation P' . If there is no solution, output -1 .

Input

The first line contains three integers n ($1 \leq n \leq 2 \times 10^5$), k , and x ($0 \leq k, x < n$).

The second line contains n integers $P_0, P_1, \dots, P_{n-1}$, representing the given permutation of numbers from 0 to $n - 1$.

Output

If there is no solution, output a single integer -1 on one line. Otherwise, output a line containing n integers, representing the valid permutation P' .

Example

standard input	standard output
4 1 3 2 0 1 3	0 3 1 2

Note

For the sample case:

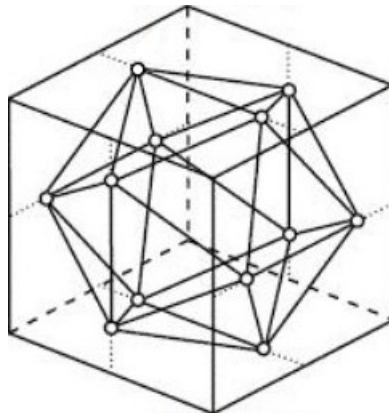
$$\begin{aligned} f(P) &= (P_1 - P_0) + (P_2 - P_0) + (P_3 - P_0) + (P_2 - P_1) + (P_3 - P_1) + (P_3 - P_2) \\ &= (0 - 2) + (1 - 2) + (3 - 2) + (1 - 0) + (3 - 0) + (3 - 1) \\ &= -2 + (-1) + 1 + 1 + 3 + 2 \\ &= 4 \equiv 0 \pmod{4} \end{aligned}$$

$$\begin{aligned} f(P') &= (P'_1 - P'_0) + (P'_2 - P'_0) + (P'_3 - P'_0) + (P'_2 - P'_1) + (P'_3 - P'_1) + (P'_3 - P'_2) \\ &= (3 - 0) + (1 - 0) + (2 - 0) + (1 - 3) + (2 - 3) + (2 - 1) \\ &= 3 + 1 + 2 + (-2) + (-1) + 1 \\ &= 4 \equiv 0 \pmod{4} \end{aligned}$$

Problem G. Precision Error?!

Input file: **standard input**
Output file: **standard output**
Time limit: 1 second
Memory limit: 1024 megabytes

Given an integer n , you need to construct a set S consisting of no more than $(2n + 2)$ different points in three-dimensional Euclidean space such that for each point $P \in S$, there are exactly n different points $Q \in S$ that $\text{dis}(P, Q) = 1$, where $\text{dis}(P, Q)$ is the Euclidean distance between P and Q .



Regular icosahedron, a feasible solution for $n = 5$.

To tolerate the precision error, let $\epsilon = 0.01$, and your answer will be considered correct if and only if:

- For any two different points $P, Q \in S$, $\text{dis}(P, Q) > \epsilon$;
- For each point $P \in S$, there are exactly n different points $Q \in S$ such that $1 - \epsilon < \text{dis}(P, Q) < 1 + \epsilon$.

Input

The first line of the input contains an integer T ($1 \leq T \leq 100$), indicating the number of test cases. For each test case:

The only line contains an integer n ($1 \leq n \leq 100$).

Output

For each test case:

Output an integer m ($1 \leq m \leq 2n + 2$) in the first line, indicating the number of points in S .

Then output m lines, each of which contains three real numbers x , y , and z ($-100 \leq x, y, z \leq 100$), indicating the coordinates of a point in S .

It can be shown that a feasible solution always exists under the given constraints, and if there are multiple feasible solutions, you may output any.

Example

standard input	standard output
5	2
1	0.000000000 0.000000000 0.000000000
2	1.000000000 0.000000000 0.000000000
3	3
4	0.000000000 0.000000000 0.000000000
5	1.000000000 0.000000000 0.000000000
	0.500000000 0.866025404 0.000000000
	4
	0.500000000 0.000000000 -0.353553391
	-0.500000000 0.000000000 -0.353553391
	0.000000000 0.500000000 0.353553391
	0.000000000 -0.500000000 0.353553391
	6
	0.707106781 0.000000000 0.000000000
	-0.707106781 0.000000000 0.000000000
	0.000000000 0.707106781 0.000000000
	0.000000000 -0.707106781 0.000000000
	0.000000000 0.000000000 0.707106781
	0.000000000 0.000000000 -0.707106781
	12
	0.000000000 0.500000000 0.809016994
	0.000000000 -0.500000000 0.809016994
	0.000000000 0.500000000 -0.809016994
	0.000000000 -0.500000000 -0.809016994
	0.500000000 0.809016994 0.000000000
	0.500000000 -0.809016994 0.000000000
	-0.500000000 0.809016994 0.000000000
	-0.500000000 -0.809016994 0.000000000
	0.809016994 0.000000000 0.500000000
	0.809016994 0.000000000 -0.500000000
	-0.809016994 0.000000000 0.500000000
	-0.809016994 0.000000000 -0.500000000

Problem H. Rock-Paper-Scissors Master

Input file: **standard input**
Output file: **standard output**
Time limit: 4 seconds
Memory limit: 1024 megabytes

Alice and Bob are playing a k -round rock-paper-scissors game. At the beginning, both players have exactly 3 cards, each card being Rock (R), Scissors (S), or Paper (P). In each round, the game proceeds as follows:

1. Both players can see all 6 cards held by each other.
2. Alice first selects one card from her hand and plays it.
3. Bob, seeing the card Alice played, chooses one of his cards to play.
4. The winner is determined by the standard rules: Rock beats Scissors, Scissors beats Paper, and Paper beats Rock. Identical cards result in a tie. If Alice wins, she scores 3; if it is a tie, she scores 1; if Bob wins, she scores 0.
5. Both players discard their played cards and independently draw a new card, which is Rock, Scissors, or Paper, with equal probability.

Alice aims to maximize her expected total score, while Bob aims to minimize Alice's expected total score. Given the number of rounds k , Alice's initial hand, and Bob's initial hand, you need to find the maximum expected total score that Alice can achieve, assuming both players play optimally.

Input

The first line of the input contains an integer T ($1 \leq T \leq 10^5$), indicating the number of test cases. For each test case:

The first line contains an integer k ($1 \leq k \leq 10^9$), indicating the number of rounds.

The second line contains a string of length 3 consisting of the characters R, S, and P, indicating Alice's initial hand.

The third line contains a string of length 3 consisting of the characters R, S, and P, indicating Bob's initial hand.

Output

For each test case, output a line containing a real number representing Alice's maximum expected total score.

Your answer is acceptable if its absolute or relative error does not exceed 10^{-6} . Formally, let your output be a and the jury's answer be b . Your output is accepted if and only if $\frac{|a-b|}{\max(1, |b|)} \leq 10^{-6}$.

Example

standard input	standard output
2	1.000000000000
1	706356363.862663573876
RRS	
PSS	
1000000000	
RSP	
RSP	

Problem I. Combination of Two Nice Problems

Input file: **standard input**
Output file: **standard output**
Time limit: **2 seconds**
Memory limit: **1024 megabytes**

Still remember Problem F. The Bond Beyond Time and Problem K. Relay Jump, the two most “notorious” problems from the 2025 ICPC Shenyang Regional Contest — one asking you to orient edges, the other sending frogs flying by reflection? If you still do, congratulations: today they return, not one at a time, but as a combo meal!



There are n frogs numbered from 1 to n . Frog i is initially located at integer coordinate x_i on the number line. Several pairs of frogs are friends, which can be abstracted as a simple, connected undirected graph with n vertices and m edges.

You must choose a direction for every undirected edge. After that, each frog independently chooses exactly one of its outgoing edges and remembers the frog at the other end as its target fixed throughout the following infinite process. In each moment of the process, all frogs jump simultaneously. If frog i 's target is frog j , then frog i lands at the reflection of its current position across the current position of frog j (if frog i coincides with frog j before the jump, then it lands at the coinciding coordinate).

Your task is to orient all edges so that every vertex has at least one outgoing edge and, no matter how the frogs choose their targets, every frog can get arbitrarily far from where it starts. Formally, for every possible target choice, for every frog i , and for every integer $N > 0$, there must exist a moment when frog i 's coordinate x'_i satisfies $|x_i - x'_i| > N$. If such an orientation does not exist, report it.

Input

The first line of the input contains an integer T ($1 \leq T \leq 10^5$), denoting the number of test cases. For each test case:

The first line contains two integers n and m ($2 \leq n \leq 2 \times 10^5$, $n - 1 \leq m \leq \min\{2 \times 10^5, \frac{n(n-1)}{2}\}$), denoting the number of frogs and the number of friendship pairs.

The second line contains n integers $x_1, x_2, \dots, x_n$ ($-10^9 \leq x_i \leq 10^9$), where x_i is the initial coordinate of frog i .

Each of the next m lines contains two integers u and v ($1 \leq u, v \leq n$), describing an undirected edge which represents the friendship between frog u and frog v . It is guaranteed that the given graph is connected and contains no self-loops or multiple edges.

It is guaranteed that both the sum of n and the sum of m do not exceed 2×10^5 .

Output

For each test case:

If it is possible to orient all edges to satisfy the problem's requirements, print "Yes" (without quotes) in the first line.

Then print m lines. Each line must contain two integers u' and v' , describing a directed edge from u' to v' . The unordered pair (u', v') must be one of the given undirected edges. You may print these directed edges in any order.

If no valid orientation exists, print a single line containing "No" (without quotes).

Example

standard input	standard output
2	Yes
4 5	2 1
1 2 3 4	1 3
1 2	4 1
1 3	2 3
1 4	3 4
2 3	No
3 4	
2 1	
4 -3	
1 2	

Note

For the first test case of the sample case, a valid orientation is to orient the edges as $2 \rightarrow 1$, $1 \rightarrow 3$, $4 \rightarrow 1$, $3 \rightarrow 2$, and $3 \rightarrow 4$. If frog 3 chooses frog 2 as its fixed target, the coordinates of all frogs change as $(1, 2, 3, 4) \rightarrow (5, 0, 1, -2) \rightarrow (-3, 10, -1, 12) \rightarrow \dots$. If frog 3 chooses frog 4 instead, the coordinates of all frogs change as $(1, 2, 3, 4) \rightarrow (5, 0, 5, -2) \rightarrow (5, 10, -9, 12) \rightarrow \dots$. It can be proved that in both cases, every frog can get arbitrarily far from where it starts.

For the second test case of the sample case, orienting the only edge as $1 \rightarrow 2$ leaves 2 with no outgoing edges, while orienting it as $2 \rightarrow 1$ leaves 1 with no outgoing edges. Thus, there is no valid orientation.

Problem J. Show Hand

Input file: standard input
Output file: standard output
Time limit: 3 seconds
Memory limit: 1024 megabytes

You are Comment allez-vous, engaged in a showdown game of Show Hand with the French gambling god Pierre Cashion. A standard deck of 52 cards is used for the game. Currently, each player has four face-up cards and one hole card. Due to the French gambling god's superior psychic ability, he can first transform his hole card into any card except the eight face-up cards already on the table. After the French gambling god has made his transformation, you are fully informed of his choice, and then you can transform your hole card into any card except the eight face-up cards already on the table and the French gambling god's transformed hole card. In other words, there are 43 cards available in total, excluding the nine known distinct cards. If you have a winning strategy, print "WoYaoYanPai"; if the French gambling god has a winning strategy, print "GeiWoCaPiXie"; if neither has a winning strategy and the game ends in a draw, print "PaiMeiYouWenTi".

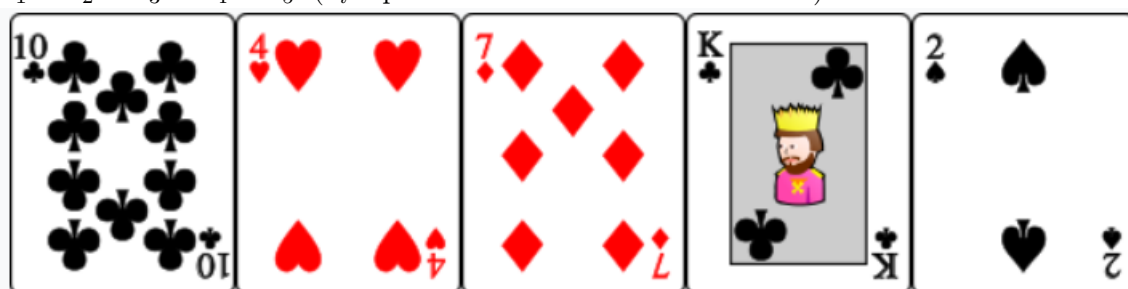
In the game of Show Hand, each player ends up with five cards in their hand, and all cards across players are unique. All cards are drawn from a standard 52-card deck. A standard 52-card deck comprises 13 ranks in each of the four French suits: clubs (♣), diamonds (♦), hearts (♥), and spades (♠). Each suit includes an Ace (A), a King (K), Queen (Q), and Jack (J), each depicted alongside a symbol of its suit; and numerals or pip cards from the Deuce (Two) to the Ten, with each card depicting that many symbols (pips) of its suit. No card can be drawn more than once.

Example set of 52 playing cards; 13 of each suit: clubs, diamonds, hearts, and spades

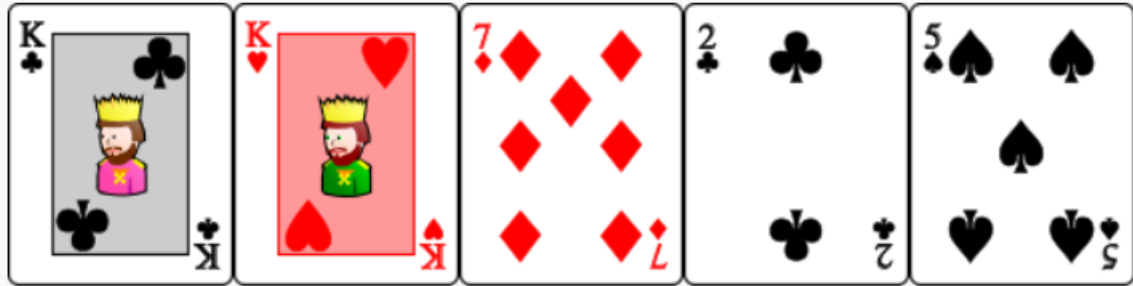
	Ace	2	3	4	5	6	7	8	9	10	Jack	Queen	King
Clubs													
Diamonds													
Hearts													
Spades													

Individual cards are ranked as follows (high-to-low): A, K, Q, J, 10, 9, 8, 7, 6, 5, 4, 3, 2. The following table shows the possible five-card poker hand types in increasing order of their values. Each type has a specific ordering of the five cards that is described below.

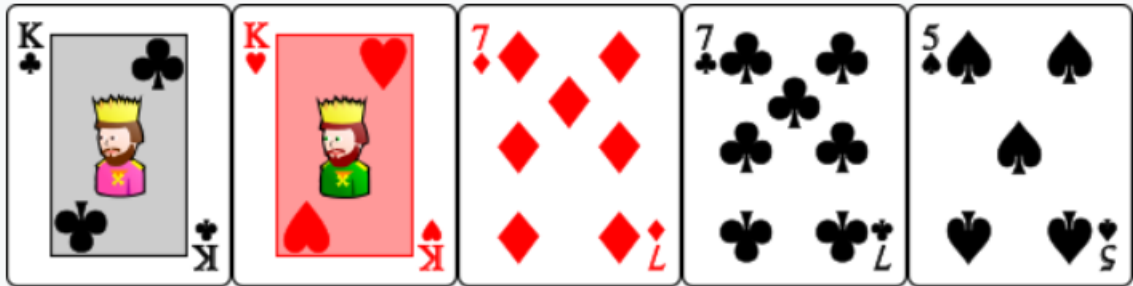
- Highcard: Simple value of the card. The cards are ordered as $a_1a_2a_3a_4a_5$ such that $a_1 > a_2 > a_3 > a_4 > a_5$. (a_i represents the rank of the i -th card.)



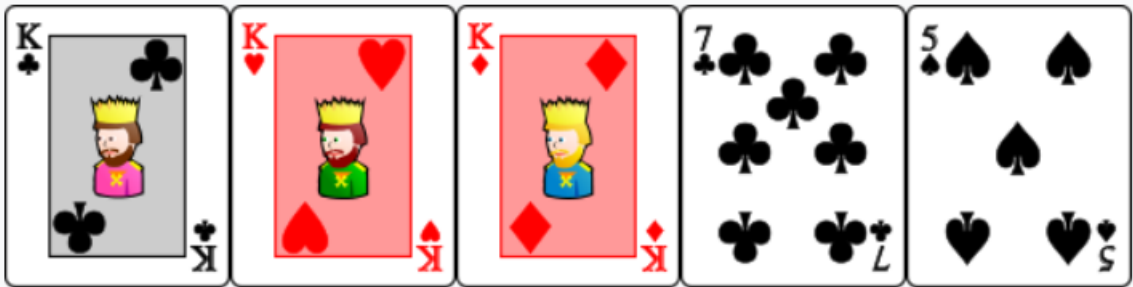
- Pair: Two cards with the same value. The cards are ordered as $a_1a_2a_3a_4a_5$ such that $a_1 = a_2$, $a_3 > a_4 > a_5$, $a_1 \neq a_3$, $a_1 \neq a_4$, $a_1 \neq a_5$.



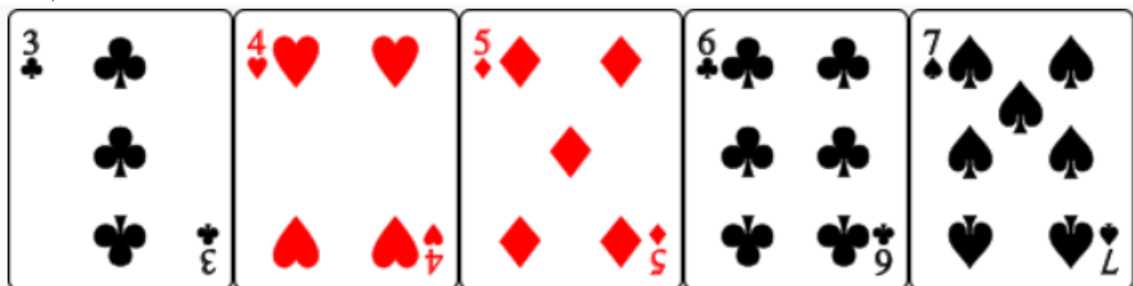
- Two pairs: Two pairs of cards with the same value. The cards are ordered as $a_1a_2a_3a_4a_5$ such that $a_1 = a_2$, $a_3 = a_4$, $a_1 > a_3$, $a_1 \neq a_5$, $a_3 \neq a_5$.



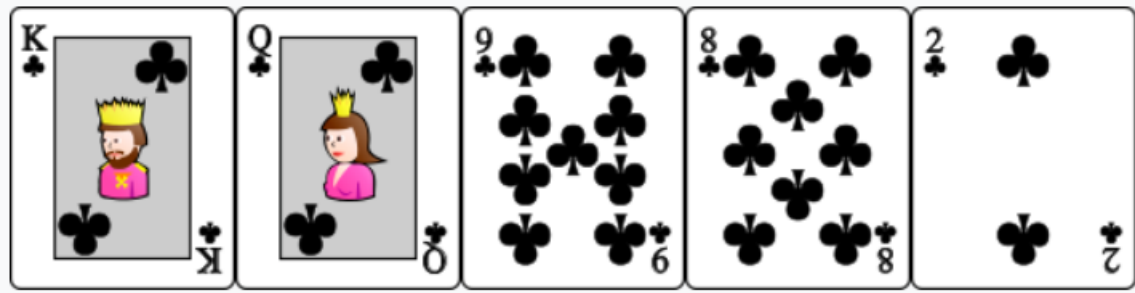
- Three of a kind: Three cards with the same value. The cards are ordered as $a_1a_2a_3a_4a_5$ such that $a_1 = a_2 = a_3$, $a_4 > a_5$, $a_1 \neq a_4$, $a_1 \neq a_5$.



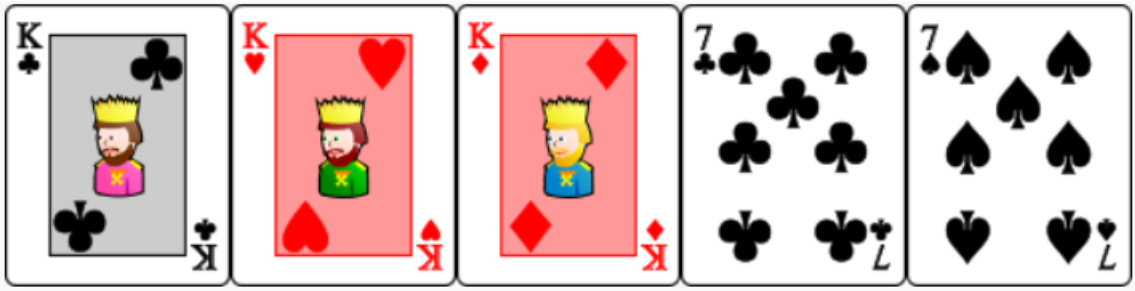
- Straight: A sequence of 5 cards in increasing value. The cards are ordered as $a_1a_2a_3a_4a_5$ such that a_i is exactly one rank above a_{i+1} for all $1 \leq i \leq 4$. Specifically, if a_5 is Ace, a_4 can be 2. In this case, Ace is considered one rank below 2.



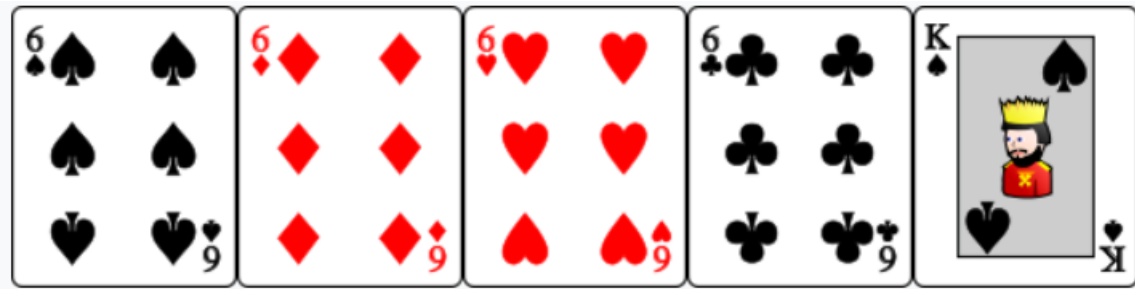
- Flush: 5 cards of the same suit. The cards are ordered as $a_1a_2a_3a_4a_5$ such that all five cards have the same suit and $a_1 > a_2 > a_3 > a_4 > a_5$.



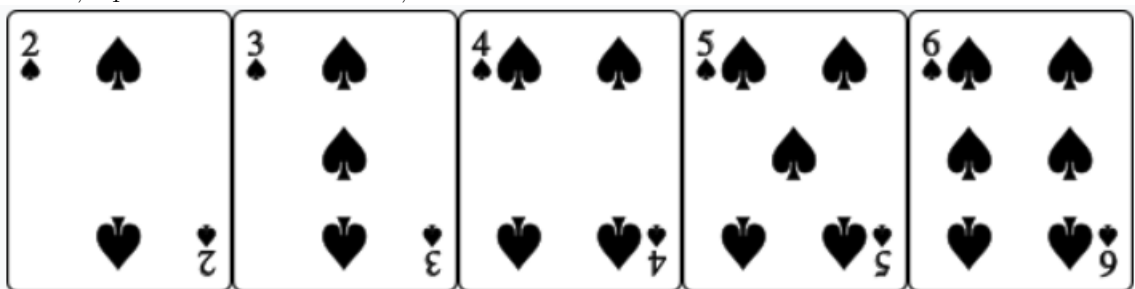
- Full house: A combination of three of a kind and a pair. The cards are ordered as $a_1a_2a_3a_4a_5$ such that $a_1 = a_2 = a_3, a_4 = a_5$.



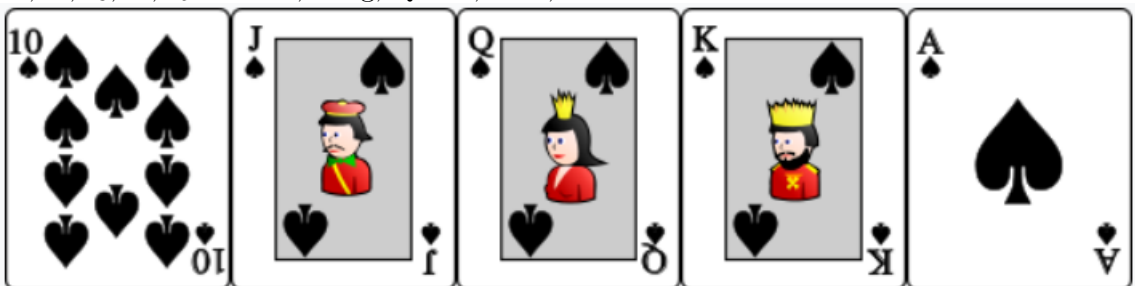
- Four of a kind: Four cards of the same value. The cards are ordered as $a_1a_2a_3a_4a_5$ such that $a_1 = a_2 = a_3 = a_4$.



- Straight flush: A straight of the same suit. The cards are ordered as $a_1a_2a_3a_4a_5$ such that all five cards have the same suit and a_i is exactly one rank above a_{i+1} for all $1 \leq i \leq 4$. Specifically, if a_5 is Ace, a_4 can be 2. In this case, Ace is considered one rank below 2.



- Royal flush: A straight flush from Ten to Ace. The cards are ordered as $a_1a_2a_3a_4a_5$ such that a_1, a_2, a_3, a_4, a_5 are Ace, King, Queen, Jack, and Ten of the same suit.



To compare two hands, first we will compare the types of the two hands. For example, if one hand is Four of a kind and the other hand is Full house, Four of a kind always wins against Full house.

If the types of the two hands are the same, we compare the ranks of the cards. We will order the cards as described above, and compare them one by one. First, we will compare the first card. If a hand's first card has a higher rank, it wins. If the first cards of the two hands have the same rank, we will compare the second card, and so on. If the cards have the same rank in every position, no one wins. The suit of the cards never matters. For example, ♣ 5, ♦ 5, ♥ 5, ♠ 2, ♣ 2 can beat ♦ 3, ♠ 3, ♥ 3, ♦ A, ♥ A. Since they are both Full house, we will compare the ranks of the three cards of a kind first.

Input

The first line of the input contains an integer T ($1 \leq T \leq 10^4$) denoting the number of test cases. For each test case:

The only line contains eight strings $c_1, c_2, c_3, c_4, p_1, p_2, p_3, p_4$ separated by single spaces, denoting the four face-up cards of you, Comment allez-vous, and the four face-up cards of the French gambling god Pierre Cashon.

For each card, the first character of its corresponding string denotes its rank. Possible ranks are 2, 3, 4, 5, 6, 7, 8, 9, T, J, Q, K, A, where T denotes 10. The second character denotes its suit: C denotes clubs, D denotes diamonds, H denotes hearts, and S denotes spades.

It is guaranteed that each card appears at most once in one test case.

Output

For each test case, print one line. If you have a winning strategy, print "WoYaoYanPai" (without quotes); if the French gambling god has a winning strategy, print "GeiWoCaPiXie" (without quotes); if neither has a winning strategy and the game ends in a draw, print "PaiMeiYouWenTi" (without quotes).

Example

standard input	standard output
3 AS KH KD AC AH KS KC AD 2D 3C 3D 2C AH QH JH 2H 4C 6H KH 9H 5H 6C 7H 9S	PaiMeiYouWenTi WoYaoYanPai GeiWoCaPiXie

Note

For the first sample case, both players can turn their hole card into a Q, forming a "AAKKQ" Two Pairs hand, ending in a draw, so neither has a winning strategy.

For the second sample case, you can always turn your hole card into any 3, forming a "33322" Full House hand, while the French gambling god can only achieve a Flush at best, so you have a winning strategy.

For the third sample case, the French gambling god can turn his hole card into any 8, forming a "56789" Straight hand, while you can only achieve a Pair at best, so the French gambling god has a winning strategy.

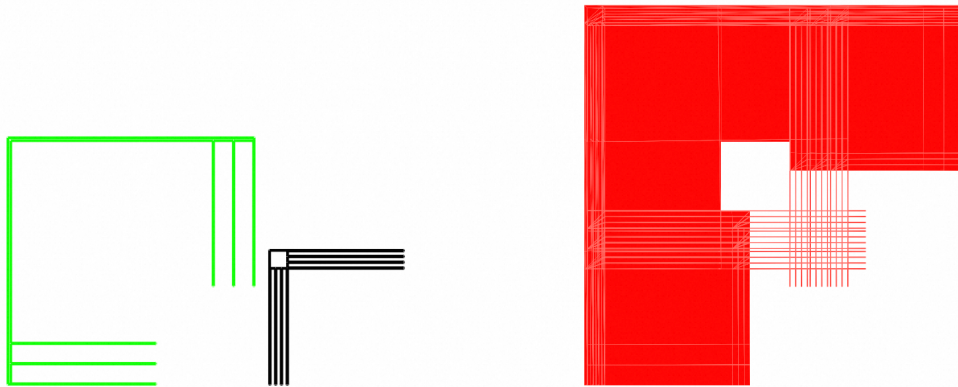
Problem K. Geometry Textbook

Input file: `standard input`
Output file: `standard output`
Time limit: 6 seconds
Memory limit: 1024 megabytes

For two subsets $A, B \subseteq \mathbb{R}^2$, the Minkowski sum of A and B is defined as:

$$A + B = \{a + b \mid a \in A, b \in B\}$$

Given two simple polygons P and Q in the plane, you need to find the area of $P + Q$. Note that each polygon is interpreted as the closed region consisting of its interior and boundary (i.e., the filled polygon).



Oks, E. (2005). Minkowski Sums of Simple Polygons. Tel-Aviv University Thesis, Fig. 1.2.

Input

The first line of the input contains an integer T ($1 \leq T \leq 10$), indicating the number of test cases. For each test case:

The first line contains two integers n and m ($3 \leq n, m \leq 30$), representing the number of vertices of the two simple polygons, respectively.

The next n lines each contain two integers x and y ($-100 \leq x, y \leq 100$), giving the vertex coordinates (x, y) of P .

The next m lines each contain two integers x and y ($-100 \leq x, y \leq 100$), giving the vertex coordinates (x, y) of Q .

It is guaranteed that the vertices of each polygon are given in counter-clockwise order. The polygons are simple, meaning their vertices are distinct and no two edges intersect or touch, except that consecutive edges meet at their common vertex. Additionally, no two consecutive edges are collinear.

It is guaranteed that both the sum of n and the sum of m over all test cases do not exceed 30.

Output

For each test case, output a single line containing a real number, representing the area of $P + Q$.

Your answer is acceptable if its absolute or relative error does not exceed 10^{-6} . Formally, let your output be a and the jury's answer be b . Your output is accepted if and only if $\frac{|a-b|}{\max(1, |b|)} \leq 10^{-6}$.

Example

standard input	standard output
2	2.000000000000
3 3	103.500000000000
0 0	
1 0	
0 1	
0 0	
1 0	
0 1	
10 3	
0 0	
10 0	
10 9	
9 8	
9 1	
1 1	
1 9	
8 9	
9 10	
0 10	
0 0	
2 0	
0 2	

Problem L. Substrings of Substrings

Input file: **standard input**
Output file: **standard output**
Time limit: 5 seconds
Memory limit: 1024 megabytes

Given a string $S = s_1s_2 \dots s_n$ of length n and a sequence $a_1, a_2, \dots, a_n$ of length n , and given q queries, each consisting of a string t :

If an interval $[l, r]$ ($1 \leq l \leq r \leq |S|$) satisfies that t is a substring of $S[l, r] = s_ls_{l+1} \dots s_r$, then this interval is called a good interval.

For all good intervals, find the maximum value of $f(l, r) = a_l + a_{l+1} + \dots + a_r$, and the sum of $f(l, r)$ over all good intervals modulo 998 244 353.

It is guaranteed that there exists at least one good interval.

Input

The first line contains two integers n ($1 \leq n \leq 10^5$) and q ($1 \leq q \leq 3 \times 10^5$), representing the length of the string and the sequence, as well as the number of queries.

The second line contains a string S of length n .

The third line contains n integers $a_1, a_2, \dots, a_n$ ($-10^9 \leq a_i \leq 10^9$), representing the given sequence.

The next q lines each contain a string t , representing a query. It is guaranteed that there exists at least one good interval.

It is guaranteed that the total length of t does not exceed 3×10^5 and that all input strings contain only lowercase English letters.

Output

Output q lines. Each line contains two integers, representing the maximum value of $f(l, r)$ in the query, and the sum of $f(l, r)$ over all good intervals modulo 998 244 353.

Example

standard input	standard output
8 2 abbabaab 3 -1 4 -1 5 -9 2 -6 bab aa	10 13 3 998244301

Note

For the first query, the good interval that achieves the maximum value of $f(l, r)$ is $[1, 5]$.

For the second query, the good interval that achieves the maximum value of $f(l, r)$ is $[1, 7]$.